Photo Editor

Project Documentation

Design, implementation and code walkthrough

Project type	Browser-based image editor (filters and presets)
Technology	HTML5, CSS3, vanilla JavaScript - no frameworks or build tools
Key browser APIs	Canvas 2D (including the filter property), File input, Object URLs, toDataURL
External resource	Remix Icon font (CDN) - used for the placeholder icon only
Files	index.html (49 lines), style.css (170), theme.css (50), script.js (433)

Contents

Contents 2

1. Introduction and Overview 3

2. Project Structure 4

3. Core Concepts and Architecture 5

4. How the Project Was Built 7

5. HTML in Detail 8

6. CSS in Detail 9

7. JavaScript in Detail 11

8. Presets Reference 16

9. Known Limitations and Recommended Improvements 17

10. Running, Customising and Testing 19

11. Glossary 21

Appendix A - Complete Source Code 22

1. Introduction and Overview

This project is a simple photo editor that works completely inside the browser. The user chooses an image from their device, adjusts nine different filters with sliders, or applies one of sixteen ready-made presets, and then downloads the edited picture as a PNG file. No image is ever uploaded to a server: the file is read locally, drawn on an HTML canvas, and processed by the browser itself.

The project is the third in a series of vanilla JavaScript projects (after a Snake game and a Kanban board). Where those projects concentrated on timers, the DOM and drag and drop, this one introduces the **Canvas API**, working with **files and images**, and **generating UI from data**.

1.1 Features

Feature	What the user sees	Main code involved
Choose image	A button opens the device's file picker (images only). The picture appears in the preview area.	`#image-input` change handler
Nine live filters	Sliders for brightness, contrast, saturation, hue rotation, blur, grayscale, sepia, opacity and invert. The picture updates while dragging.	`filters` object, `createFilters()`, `applyFilters()`
Sixteen presets	One-click looks such as Vintage, Noir, Cyberpunk and Dreamy.	`presets` object and preset buttons
Reset	Returns every filter to its default value.	Reset click handler
Download	Saves the edited picture as edited-image.png at the original resolution.	`canvas.toDataURL()`
Empty state	Before any image is chosen, a large icon and the text "No Image Chosen" are shown.	.placeholder element, CSS

1.2 Technology stack

Technology	Used for	Reason
HTML5	Layout skeleton, file input, canvas element, empty containers for sliders and presets	A hidden file input inside a label gives a custom-styled upload button with no JavaScript.
CSS3 (custom properties, Flexbox)	Dark theme, two-column layout, custom slider styling	Design tokens live in a separate theme.css so the look can be changed in one file.
Vanilla JavaScript (ES6)	State, UI generation, image loading, filtering, export	No dependencies; shows how the browser's own APIs do the heavy work.
Canvas 2D API	Showing and processing the image	`ctx.filter` accepts the same filter syntax as CSS, so no pixel-by-pixel code is needed.
Remix Icon (CDN)	The image icon in the placeholder	Lightweight icon font loaded from a link tag.

2. Project Structure

```
photo-editor/
 |-- index.html   page markup: buttons, preview area, filter + preset panels
 |-- theme.css    design tokens (colours, spacing, font sizes, radii) as CSS variables
 |-- style.css    layout and component styling, uses the tokens from theme.css
 |-- script.js    state, slider/preset generation, image loading, filtering, export
```

File	Role	Depends on
index.html	Defines the fixed parts of the page. The filter sliders and preset buttons are *not* written here; JavaScript creates them inside the empty `.filters` and `.presets` containers.	theme.css, style.css, script.js, Remix Icon CDN
theme.css	Declares CSS custom properties only - no rules that style elements.	None
style.css	Styles the layout, buttons, panels, sliders, canvas and presets.	Variables from theme.css; class and id names from index.html
script.js	All behaviour.	Ids and classes from index.html (`#image-canvas`, `#image-input`, `#reset-btn`, ...)

The stylesheet order in the HTML head is theme.css first, then style.css, which reads naturally (tokens before the rules that use them). The script tag is at the end of the body, so every element it selects already exists when it runs.

2.1 Page layout

The page is one horizontal flex row with two columns. The left column (70% width) holds the three buttons and the preview box. The right column (30%) is a panel with the filter sliders on top and the preset buttons below.

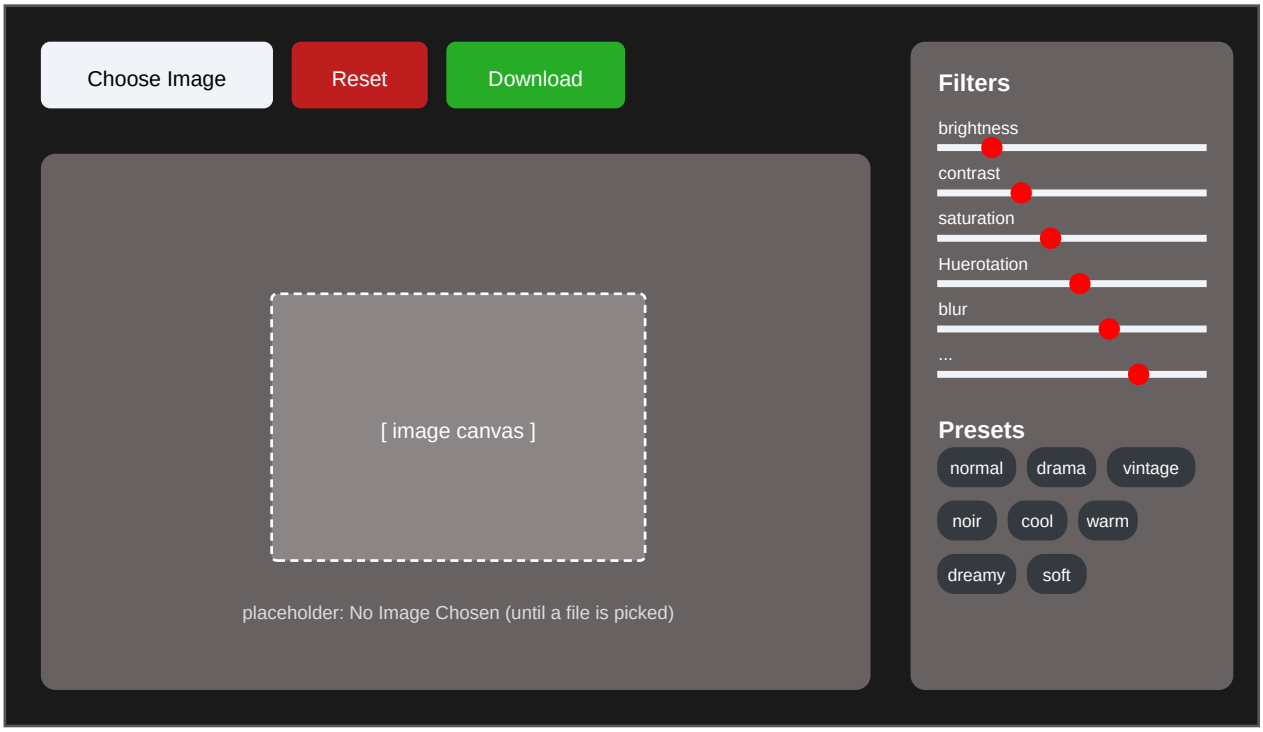


Figure 1 - Layout of the editor

3. Core Concepts and Architecture

3.1 Non-destructive editing

The most important design idea is that the **original image is never changed**. When a file is loaded, it is stored in the variable `image` as an `Image` object. Every time a filter changes, the canvas is cleared and the *original* image is drawn again with the complete set of current filter values. This means filters never pile up on top of each other: moving the brightness slider from 120 to 100 gives back exactly the original brightness. It is also why Reset works so simply - set the numbers back and redraw.

3.2 The three pieces of state

State	Variable	Description
Filter values	`filters`	An object with one entry per filter. Each entry stores the current `value`, the slider `min` and `max`, and the CSS `unit`. This is the single source of truth for every edit.
Original picture	`image`	The loaded `Image` element. Starts as `null` and is checked in `applyFilters()` so nothing happens before an image exists.
Display	The `<canvas>` element	The visible result. It is a render target that is fully redrawn on each change - not a place where data is kept.

3.3 How Canvas filters work

Normally, editing an image with a canvas means reading its pixels and changing them in a loop. This project avoids that by using `canvasCtx.filter`, a property of the 2D drawing context that takes a string in the same format as the CSS `filter` property, for example `brightness(120%)` `contrast(90%)`. Whatever is drawn after the property is set is passed through those filters by the browser, which is fast because it can use the graphics hardware.

The filters are written one after another in a single string and are applied **in that order**, each one working on the result of the previous one. This project always uses the order below. Because the order is fixed, a given set of slider values always produces the same picture.

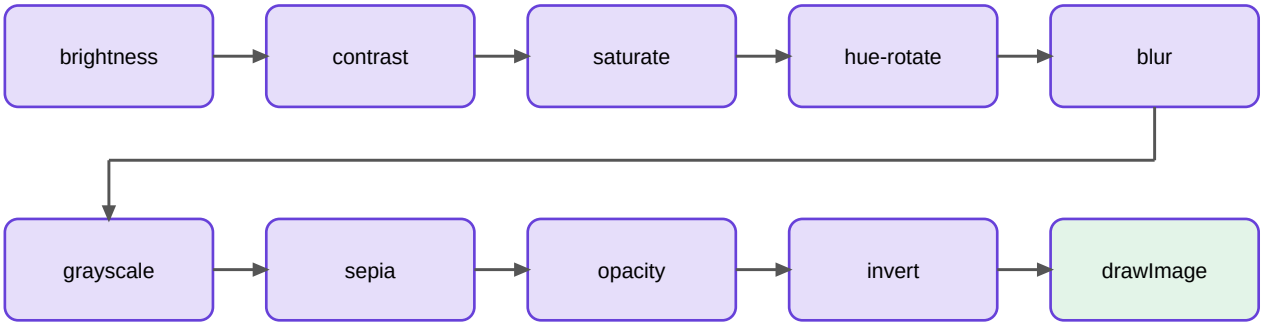


Figure 2 - The filter chain built in applyFilters()

3.4 Data flow

There are three flows through the program: loading an image, editing it, and exporting it.

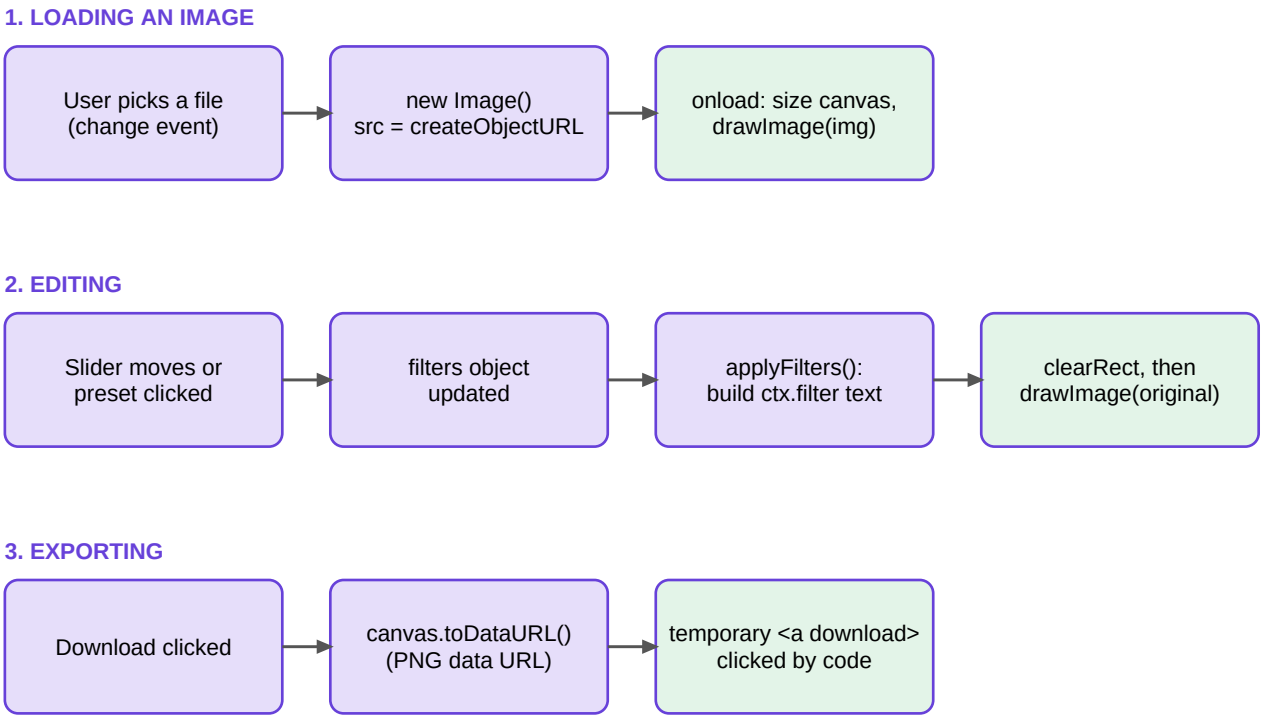


Figure 3 - Loading, editing and exporting

4. How the Project Was Built

The project was developed in layers, getting something visible on the screen as early as possible, then adding one capability at a time. The steps below reconstruct that process from the final code.

Step 1 - Plan

The goal was a tool with four abilities: pick an image, adjust filters, apply presets, save the result. The key technical decision was to use the canvas filter property instead of manual pixel editing, and to describe the filters as data (an object) so the sliders could be generated rather than written by hand nine times.

Step 2 - HTML skeleton

- A `<main><section>` with a `.left` column (toolbar and preview) and a `.right` column (panel).
- The upload control is a `<label for="image-input" class="btn">` wrapping a hidden file input. Clicking the label opens the file picker, so the button can be styled freely.
- The preview box contains both the placeholder and the canvas; JavaScript later swaps which one is visible.
- The `.filters` and `.presets` containers are left empty on purpose.

Step 3 - Design tokens (theme.css)

Before styling anything, a theme file defined the palette and scales: background and text colours, padding, margin and gutter sizes, font sizes and border radii. Writing these as CSS variables means style.css never contains raw colour or spacing values.

Step 4 - Layout and components (style.css)

1. Reset, base font and the dark page colours.
2. Two-column flex layout with 70/30 widths.
3. Buttons: a shared `.btn` style, with red (reset) and green (download) variants.
4. The preview box with a fixed 5:3 aspect ratio, centring its contents.
5. The placeholder (large icon, dimmed) and a canvas that is hidden until an image exists.
6. Custom range-slider styling (thin track, round red thumb).
7. Wrapping preset chips.

Step 5 - JavaScript, one feature at a time

1. Define the `filters` object (names, defaults, limits, units).
2. Write `createFilterElement()` and `createFilters()` to generate the sliders from that object.
3. Handle the file input: read the file, create an `Image`, draw it on the canvas.
4. Write `applyFilters()` to build the filter string and redraw, and connect it to the slider `input` event.
5. Add Reset (restore defaults and rebuild the sliders) and Download (export the canvas).

6 Add the `presets` object and generate a button for each preset.

Step 6 - Test

Testing was done by hand with different images, moving each slider through its range, trying every preset and checking the downloaded file. A checklist is in Section 10.

5. HTML in Detail

```
<main>
  <section>
    <div class="left">
      <div class="top">
        <label for="image-input" class="btn">Choose Image
          <input id="image-input" type="file" accept="image/*" hidden />
        </label>
        <button class="btn" id="reset-btn">Reset</button>
        <button class="btn" id="download-btn">Download</button>
      </div>
      <div class="bottom">
        <div class="placeholder">
          <i class="ri-image-line"></i>
          <p>No Image Chosen</p>
        </div>
        <canvas id="image-canvas"></canvas>
      </div>
    </div>
    <div class="right">
      <h2>Filters</h2>   <div class="filters"></div>
      <h2>Presets</h2>   <div class="presets"></div>
    </div>
  </section>
</main>
```

Element	Purpose	Used by
`label.btn[for=image-input]`	The visible 'Choose Image' button. The `for` attribute links it to the file input, so clicking it opens the picker.	CSS (.btn)
`#image-input`	Hidden file input restricted to images with `accept="image/*"`.	JS `change` event
`#reset-btn`, `#download-btn`	Action buttons.	JS click handlers; CSS colour overrides
`.placeholder`	Empty-state icon and message. Hidden by JS when an image is chosen.	JS `style.display`; CSS
`#image-canvas`	The drawing surface that shows the edited picture.	JS (context, size, export); CSS (hidden by default)
`.filters`	Empty container that receives the nine generated slider groups.	`createFilters()`
`.presets`	Empty container that receives the generated preset buttons.	Preset loop at the end of script.js
`i.ri-image-line`	Icon from the Remix Icon font.	CDN stylesheet in the head

Note: the icon comes from an external stylesheet (cdn.jsdelivr.net). If the page is opened offline, the placeholder text still shows but the icon will not. Everything else works offline.

6. CSS in Detail

6.1 theme.css - design tokens

This file contains a single `:root` rule that declares CSS custom properties, used everywhere else with `var(--name)`.

Group	Variables	Values
Background colours	`--bg-primary-color, --bg-secondary-color, --bg-active-color, --bg-danger-color, --bg-success-color, --preset-button-color`	near-black, grey, off-white, red, green, dark slate (#343a40)
Text colours	`--text-primary-color, --text-secondary-color, --text-tertiary-color, --text-active-color`	white, grey, off-white, black
Spacing scales	`--padding-*, --margin-*, --gutter-*`	sm to 2xl: 0.5, 1, 1.5, 2 and 3 rem
Font sizes	`--font-size-sm ... --font-size-2xl`	0.875 to 2 rem
Border radius	`--border-radius-sm ... --border-radius-2xl`	0.2 to 0.5 rem

Using rem units makes the whole interface scale with the user's base font size. Some tokens (for example the margin variants beyond `--margin-sm`, the secondary and tertiary text colours and the larger font sizes) are defined for future use and are not currently referenced.

6.2 style.css - layout

Rule	Effect
`main section { display:flex; padding:3rem; gap:1.5rem; }`	Puts the left and right columns side by side with spacing.
`.left { width:70%; display:flex; flex-direction:column; align-items:center; }`	Toolbar above preview, centred, with a gap.
`.right { width:30%; background:secondary; padding; border-radius; display:flex; flex-direction:column; }`	The side panel; its headings, filter list and presets stack vertically.
`.top { display:flex; gap }`	The three buttons in a row.
`.bottom { width:100%; aspect-ratio:5/3; display:flex; justify-content:center; align-items:center; }`	The preview box keeps a 5:3 shape and centres whatever it holds (placeholder or canvas).

6.3 Buttons

A shared `.btn` rule gives padding, no border or outline, rounded corners, a pointer cursor and the light 'active' colours. `#reset-btn` and `#download-btn` override the background with the danger (red) and success (green) tokens and switch the text to white. The ids are more specific than the class, so the overrides win.

6.4 Placeholder and canvas

- The placeholder is a dimmed (`opacity: 0.6`) column with a very large icon (`font-size: 10rem`) and a caption.
- `canvas` starts with `display: none`. When an image is chosen, JavaScript sets it to `block` and hides the placeholder.
- `max-width: 90%` and `max-height: 80vh` shrink large photos to fit the screen visually. The canvas keeps its full internal resolution; only its on-screen size changes.

6.5 Custom slider

```
.filter input { width:100%; outline:none; margin-top:var(--margin-sm); appearance:none; }

.filter input::-webkit-slider-runnable-track {
  background-color: var(--bg-active-color);  height: 3px;  border-radius: var(--border-radius-sm);
}
.filter input::-moz-range-track { background-color: var(--bg-active-color); height: 3px; }

.filter input::-webkit-slider-thumb {
  cursor: pointer;  appearance: none;
  width: 20px;  height: 20px;  border-radius: 50%;
  background-color: red;  margin-top: -8.5px;
}
```

Browsers draw range inputs with their own look, so the default is switched off with `appearance: none` and the parts are restyled through vendor pseudo-elements: the **track** (a thin light line) and the **thumb** (a red circle). The thumb's negative top margin centres the 20 px circle on the 3 px track in WebKit/Blink browsers. Only a track rule exists for Firefox (`::-moz-range-track`), so Firefox shows its default thumb.

6.6 Presets

Preset chips are laid out with `display:flex; flex-wrap:wrap` and a small gap, so they flow onto new lines. Each `.preset-btn` has a dark slate background, white text and a large radius.

7. JavaScript in Detail

7.1 The filters object

```
let filters = {
  brightness:  { value: 100, min: 0, max: 200, unit: "%"   },
  contrast:    { value: 100, min: 0, max: 200, unit: "%"   },
  saturation:  { value: 100, min: 0, max: 200, unit: "%"   },
  Huerotation: { value: 0,   min: 0, max: 360, unit: "deg" },
  blur:        { value: 0,   min: 0, max: 20,  unit: "px"  },
  grayscale:   { value: 0,   min: 0, max: 100, unit: "%"   },
  sepia:       { value: 0,   min: 0, max: 100, unit: "%"   },
  opacity:     { value: 100, min: 0, max: 100, unit: "%"   },
  invert:      { value: 0,   min: 0, max: 100, unit: "%"   },
};
```

(The source writes some entries over several lines; the content is the same.) This object describes the filters as **data**. Each entry holds everything needed to create its slider (`min`, `max`, starting `value`) and to build its CSS text (`unit`). Because the interface and the filter string are both generated from this one object, adding a new filter later mostly means adding one line here.

Filter	CSS function	Default	Slider range	Effect
brightness	`brightness()`	100%	0 - 200%	Lightens or darkens. 100% is unchanged, 0% is black.
contrast	`contrast()`	100%	0 - 200%	Difference between light and dark areas.
saturation	`saturate()`	100%	0 - 200%	Colour intensity. 0% is grey, above 100% is more vivid.
Huerotation	`hue-rotate()`	0deg	0 - 360deg	Rotates all colours around the colour wheel.
blur	`blur()`	0px	0 - 20px	Gaussian blur radius in image pixels.
grayscale	`grayscale()`	0%	0 - 100%	Converts toward black and white.
sepia	`sepia()`	0%	0 - 100%	Warm brown, old-photo tone.
opacity	`opacity()`	100%	0 - 100%	Transparency; shows as transparent areas in the PNG.
invert	`invert()`	0%	0 - 100%	Colour negative.

7.2 Setup: DOM references and variables

```
const imageCanvas   = document.querySelector("#image-canvas");
const imageInput    = document.querySelector("#image-input");
const canvasCtx     = imageCanvas.getContext("2d");
const resetBtn      = document.querySelector("#reset-btn");
const downloadBtn   = document.querySelector("#download-btn");
let file  = null;      // the File chosen by the user
let image = null;      // the loaded Image element (the untouched original)

const filersContainer  = document.querySelector(".filters");
const presetsContainer = document.querySelector(".presets");
```

getContext("2d") returns the object used for all drawing on the canvas. It is stored once as `canvasCtx` and reused. The variables `file` and `image` start as `null`, meaning 'nothing loaded yet'.

7.3 Generating the sliders

```
function createFilterElement(name, unit = "%", value, min, max) {
  const div = document.createElement("div");
  div.classList.add("filter");

  const input = document.createElement("input");
  input.type = "range";
  input.min = min;   input.max = max;   input.value = value;
  input.id = name;

  const p = document.createElement("p");
  p.innerText = name;

  div.appendChild(p);
  div.appendChild(input);

  input.addEventListener("input", (event) => {
    filters[name].value = input.value;
    applyFilters();
  });
  return div;
}
```

1 Create a wrapper `div.filter`.

2 Create a range input and give it the limits and starting value from the `filters` entry. The input's id is the filter's name, which later makes it easy to find.

3 Create a paragraph with the filter's name as its label.

4 Add both to the wrapper (label first, slider second).

5 On the `input` event - which fires continuously while the slider is dragged, unlike `change` which fires only on release - store the new value in `filters` and redraw.

6 Return the finished element to the caller.

The `input.value` is a string, so `filters[name].value` becomes a string after the first move. This works because the value is later pasted into a text string anyway.

```
function createFilters() {
  Object.keys(filters).forEach((key) => {
    const filterElement = createFilterElement(
      key, filters[key].unit, filters[key].value, filters[key].min, filters[key].max,
    );
    filersContainer.appendChild(filterElement);
  });
}
createFilters();
```

`Object.keys(filters)` lists the nine names. For each one, a slider group is built and added to the panel. The final call runs once at start-up, so the sliders exist from the first moment.

7.4 Loading an image

```
imageInput.addEventListener("change", (event) => {
  file = event.target.files[0];
  const imagePlaceholder = document.querySelector(".placeholder");
  imageCanvas.style.display = "block";
  imagePlaceholder.style.display = "none";

  const img = new Image();
  img.src = URL.createObjectURL(file);

  img.onload = () => {
    image = img;
    imageCanvas.width  = img.width;
    imageCanvas.height = img.height;
    canvasCtx.drawImage(img, 0, 0);
  };
});
```

1 When the user picks a file, take the first (only) one from `event.target.files`.

2 Show the canvas and hide the placeholder.

3 Create an `Image` and point its `src` to `URL.createObjectURL(file)`: a temporary local address that refers to the file in memory. Nothing is uploaded.

4 Images load asynchronously, so the real work is in `onload`, which runs when the pixels are ready.

5 Save the image as the original (`image = img`).

6 Set the canvas's internal width and height to the image's natural size, so no detail is lost.

7 Draw the image at the top-left corner.

Good to know: assigning `canvas.width` or `height` resets the whole drawing context, including any `filter` that was set. A newly loaded image is therefore drawn *without* filters, even if the sliders are not at their defaults (see Section 9).

7.5 applyFilters() - the heart of the program

```
function applyFilters() {
  if (!image) return;

  canvasCtx.clearRect(0, 0, imageCanvas.width, imageCanvas.height);

  canvasCtx.filter = `
    brightness(${filters.brightness.value}${filters.brightness.unit})
    contrast(${filters.contrast.value}${filters.contrast.unit})
    saturate(${filters.saturation.value}${filters.saturation.unit})
    hue-rotate(${filters.Huerotation.value}${filters.Huerotation.unit})
    blur(${filters.blur.value}${filters.blur.unit})
    grayscale(${filters.grayscale.value}${filters.grayscale.unit})
    sepia(${filters.sepia.value}${filters.sepia.unit})
    opacity(${filters.opacity.value}${filters.opacity.unit})
    invert(${filters.invert.value}${filters.invert.unit})
  `;

  canvasCtx.drawImage(image, 0, 0);
}
```

1 **Guard:** if no image is loaded, stop immediately.

2 **Clear** the canvas so nothing from the previous frame remains (important when opacity is below 100%, because transparent pixels would otherwise show the old image underneath).

3 **Build the filter string** with a template literal. Each line combines a CSS function name, the number from `filters` and its unit, for example `brightness(120%)`. The line breaks are just whitespace and are allowed in the filter syntax.

4 **Draw the original image.** Because the filter property is set, the browser applies the whole chain while drawing.

Note that `applyFilters()` takes no arguments: it reads everything it needs from the global `filters` and `image`. That is why any code that changes the data - a slider, a preset, or Reset - only needs to change `filters` and then call it.

7.6 Reset

```
resetBtn.addEventListener("click", () => {
  filters = {
    brightness: { value: 100, min: 0, max: 200, unit: "%" },
    ... (the same nine entries as the original object) ...
  };

  applyFilters();

  filersContainer.innerHTML = "";
  createFilters();
});
```

Reset replaces the `filters` object with a fresh copy of the defaults, redraws the image, empties the slider container and rebuilds all nine sliders so that they show their default positions again. This is why `filters` is declared with `let` rather than `const` - it is reassigned here.

7.7 Download

```
downloadBtn.addEventListener("click", () => {
  const link = document.createElement("a");
  link.download = "edited-image.png";
  link.href = imageCanvas.toDataURL();
  link.click();
});
```

`toDataURL()` converts what is currently on the canvas into a PNG encoded as a long text string (a data URL). A temporary link element is given that string as its address and a `download` attribute with the file name, then `click()` is called by code, which makes the browser save the file. Because the canvas has the original image's size, the downloaded file has the original resolution, regardless of how small the picture looks on screen.

7.8 Presets

A preset is a named object that stores a value for each of the nine filters. The code at the end of script.js contains sixteen of them; the full list is in Section 8.

```
const presets = {
  normal:  { brightness: 100, contrast: 100, saturation: 100, Huerotation: 0,
             blur: 0, grayscale: 0, sepia: 0, opacity: 100, invert: 0 },
  vintage: { brightness: 105, contrast: 90, saturation: 75, Huerotation: 350,
             blur: 0.3, grayscale: 10, sepia: 35, opacity: 100, invert: 0 },
  // ... 14 more
};

Object.keys(presets).forEach((presetName) => {
  const presetbtn = document.createElement("button");
  presetbtn.classList.add("preset-btn");
  presetbtn.innerText = presetName;
  presetsContainer.appendChild(presetbtn);

  presetbtn.addEventListener("click", () => {
    const preset = presets[presetName];
    Object.keys(preset).forEach((filterName) => {
      filters[filterName].value = preset[filterName];
    });
    applyFilters();
  });
});
```

1 Loop over the preset names; for each, create a button, give it the class `preset-btn` and use the preset's key as its text.

2 On click, copy each of the preset's nine numbers into the matching entry of `filters`.

3 Redraw with `applyFilters()`.

Like the sliders, the preset buttons are generated from data, so adding a new look is just adding a new object - no HTML or extra code is needed. The file also contains an empty function `createPresetElement(name, preset) {}` that is never called; it looks like a placeholder from an earlier plan to build the preset buttons in a function like the sliders.

7.9 Order of execution

1 The browser loads the HTML, then theme.css and style.css, and the icon font.

2 script.js runs: it defines `filters`, grabs the DOM elements and defines functions.

3 `createFilters()` builds the nine sliders.

4 Event listeners are attached (file input, Reset, Download).

5 The `presets` object is defined and the sixteen preset buttons are created.

6 The editor now waits for the user. Nothing is drawn until an image is chosen.

7.10 User flows

Flow	Sequence
Open an image	Click Choose Image → pick a file → `change` fires → canvas shown, placeholder hidden → image loads → canvas sized → image drawn unfiltered.
Adjust a slider	`input` fires continuously → `filters[name].value` updated → `applyFilters()` clears and redraws the original with all nine filters.
Apply a preset	Click a preset → nine values copied into `filters` → `applyFilters()` redraws.
Reset	Click Reset → `filters` replaced with defaults → redraw → sliders rebuilt at default positions.
Download	Click Download → canvas converted to a PNG data URL → temporary link clicked → browser saves edited-image.png.

8. Presets Reference

All sixteen presets and their values, in the order they appear in the code and on the page. Brightness, contrast, saturation, grayscale, sepia, opacity and invert are percentages; hue is in degrees; blur is in pixels. Opacity is 100 and invert is 0 in every preset, so presets only change the first seven filters.

Preset	Bright	Contr.	Satur.	Hue	Blur	Gray	Sepia	Opac.	Invert
`normal`	100	100	100	0	0	0	0	100	0
`drama`	90	145	125	0	0	10	5	100	0
`vintage`	105	90	75	350	0.3	10	35	100	0
`oldSchool`	95	110	65	0	0	25	45	100	0
`cinematic`	85	130	85	345	0	15	10	100	0
`blackAndWhite`	105	130	0	0	0	100	0	100	0
`faded`	115	80	70	0	0	5	15	100	0
`cool`	100	110	110	190	0	0	0	100	0
`warm`	105	105	115	350	0	0	20	100	0
`moody`	75	135	70	210	0	15	5	100	0
`fadedFilm`	110	85	80	5	0.2	10	25	100	0
`dreamy`	115	90	110	340	1	0	5	100	0
`noir`	80	160	0	0	0	100	0	100	0
`retro`	105	95	85	10	0	15	30	100	0
`cyberpunk`	95	140	160	280	0	0	0	100	0
`soft`	110	90	95	0	0.5	0	0	100	0

8.1 Grouping the presets by character

Look	Presets	Typical settings
Black and white	`blackAndWhite`, `noir`	Saturation 0, grayscale 100; noir is darker with higher contrast.
Warm / aged	`vintage`, `oldSchool`, `retro`, `warm`, `fadedFilm`	Sepia 20-45, reduced saturation, slight hue shift.
Faded / soft	`faded`, `soft`, `dreamy`	Higher brightness, lower contrast, small blur.
Dramatic / dark	`drama`, `cinematic`, `moody`	Lower brightness, contrast 130-145.
Colour-shifted	`cool`, `cyberpunk`	Large hue rotation (190 and 280 degrees); cyberpunk saturates strongly.
Reference	`normal`	All defaults - equivalent to Reset.

9. Known Limitations and Recommended Improvements

The editor works for its main purpose. Reading the code closely also shows several behaviours that are worth fixing. They are grouped by importance.

9.1 Important

Issue	What happens	Recommended fix
Presets do not move the sliders	Clicking a preset changes `filters` and the picture, but the slider thumbs stay where they were, so the UI no longer matches the state. Moving one slider afterwards changes only that filter; the other eight keep the preset's values even though their sliders show something else.	After applying a preset, update every slider: `document.getElementById(name).value = value` (ids equal filter names), or rebuild the sliders.
A new image does not reuse the current filters	Resizing the canvas resets its context, so the new picture appears unfiltered while the sliders still show old values.	Call `applyFilters()` at the end of `img.onload` (or reset the filters and sliders).
Cancelling the file picker throws an error	With no file, `files[0]` is `undefined` and `URL.createObjectURL(undefined)` raises a TypeError. The UI has also already switched to the empty canvas.	Start the handler with `if (!file) return;` and only swap placeholder/canvas inside `onload`.
No error handling for bad files	A file that cannot be decoded as an image leaves a blank canvas and no message.	Add `img.onerror` to show a message and restore the placeholder.
Presets with decimals vs slider step	Blur values such as 0.2, 0.3, 0.5 are used by presets, but a range input defaults to `step=1`, so these cannot be shown on the slider.	Set `input.step = 0.1` for blur (or all filters).

9.2 Code quality

Observation	Suggested improvement
The default filters are written twice - at the top and inside Reset.	Create one function `const defaultFilters = () => ({...})` and use it for both.
The same nine-line filter string is written by hand.	Build it from the object: `Object.entries(filters).map(...)` with a small table mapping each key to its CSS function name.
`createFilterElement` accepts `unit` but never uses it; the `event` parameter is unused; `createPresetElement` is an empty stub.	Remove unused parameters and dead code.
Names: `filersContainer` (typo), `Huerotation` (inconsistent camelCase) which is also shown as the visible label.	Rename, and store a separate human-readable `label` such as "Hue rotation".
Download uses `toDataURL()`, which creates a very large string for big photos.	Use `canvas.toBlob()` with `URL.createObjectURL` and revoke the URL afterwards.
Object URLs from loaded images are never released.	Call `URL.revokeObjectURL(img.src)` inside `onload`.
Every slider movement redraws the full-resolution image, which can lag on large photos with blur.	Throttle with `requestAnimationFrame` or edit a downscaled preview and export the full image on Download.

9.3 Interface and accessibility

- No numeric value is shown next to each slider, so users cannot see exact numbers.
- Download works even with no image, producing a blank transparent PNG. Disable the button until an image is loaded.
- The preset labels are raw code names (`blackAndWhite`, `fadedFilm`); preset buttons also lack a pointer cursor and a visible focus style, and the sliders have `outline: none` with no replacement focus indicator.
- Slider names are paragraphs, not `<label>` elements, so screen readers do not link them to the inputs.
- Firefox shows its default slider thumb, because only the WebKit thumb is styled; the thumb colour is a hard-coded `red` instead of a theme variable.
- The layout is a fixed 70/30 split with no media queries, so it is cramped on phones.
- Blur on a canvas can fade the image edges, because pixels outside the picture count as transparent.
- `CanvasRenderingContext2D.filter` has historically not been supported in every browser (notably Safari). Check the current browser-compatibility table before relying on it.

9.4 Example fix for the first two problems

```
function syncSliders() {
  Object.keys(filters).forEach((name) => {
    document.getElementById(name).value = filters[name].value;
  });
}

// in the preset click handler:
Object.keys(preset).forEach((n) => { filters[n].value = preset[n]; });
syncSliders();      // sliders now match the preset
applyFilters();

// in the image loader:
imageInput.addEventListener("change", (event) => {
  file = event.target.files[0];
  if (!file) return;                      // user cancelled
  const img = new Image();
  img.onload = () => {
    image = img;
    imageCanvas.width = img.width;  imageCanvas.height = img.height;
    imageCanvas.style.display = "block";
    document.querySelector(".placeholder").style.display = "none";
    applyFilters();                       // keep the current look
  };
  img.src = URL.createObjectURL(file);
});
```

9.5 Feature ideas

- Before/after comparison (hold a button to show the original).
- Crop, rotate and flip; resize before download.
- Choose the export format (JPEG with quality, WebP).
- Save your own presets; undo/redo history.
- Drag-and-drop or paste an image.

10. Running, Customising and Testing

10.1 How to run

1. Keep index.html, theme.css, style.css and script.js together in one folder.
2. Open index.html in a modern browser. No server or build step is required. An internet connection is needed only for the placeholder icon.
3. Click *Choose Image* and select a picture.
4. Move the sliders or click presets; click *Reset* to go back to the defaults.
5. Click *Download* to save the result as edited-image.png.

10.2 Customisation

To change	Where
Colours, spacing, radii, font sizes	CSS variables in theme.css.
A filter's range or default	Its entry in the `filters` object (and in the Reset copy).
Add a new preset	Add an object to `presets` with all nine keys. A button is created automatically.
Add a new filter	Add an entry to `filters` (and to Reset), add its function to the string in `applyFilters()`, and add its key to every preset. The slider is generated automatically. Canvas filter functions available include `drop-shadow()`.
Output file name or format	The `link.download` name and the argument of `toDataURL()` (for example `"image/jpeg", 0.9`).
Preview box shape	`aspect-ratio: 5/3` in the `.bottom` rule of style.css.

10.3 Manual test checklist

#	Test	Expected result
1	Open the page	Placeholder icon and 'No Image Chosen'; nine sliders and sixteen preset buttons visible
2	Choose a JPG, then a PNG	Image appears undistorted; placeholder disappears
3	Drag each slider from min to max	Picture changes continuously; no changes pile up when returning to the default
4	Set contrast very high, then lower it	Original contrast returns exactly (non-destructive)
5	Click each preset	Picture changes. Currently the sliders do not move (see 9.1)
6	Click Reset after a preset	Original picture returns; sliders return to default positions
7	Set opacity to 50, then download	PNG has semi-transparent pixels
8	Download a large photo	File has the original pixel dimensions
9	Open the file picker and cancel	Currently throws a console error (see 9.1)
10	Click Download before choosing an image	Currently saves a blank PNG (see 9.3)

11. Glossary

Term	Meaning
Canvas	An HTML element that provides a bitmap surface on which JavaScript can draw.
2D context	The object returned by `getContext("2d")` that holds the drawing methods and settings such as `filter`.
CSS filter	A list of effects like `blur(5px)` or `sepia(40%)` applied to graphics.
Object URL	A temporary address, created by `URL.createObjectURL`, that points to a file held in memory.
Data URL	A file encoded as text and used like a link; `toDataURL` produces one.
Template literal	A backtick string that may span lines and contain `${...}` placeholders.
Non-destructive editing	Keeping the original unchanged and re-applying all edits to it each time.
Range input	A slider control: `<input type="range">`.
Design tokens	Named values (colours, spacing) defined once and reused, here as CSS custom properties.

Appendix A - Complete Source Code

The four project files are reproduced in full. Line numbers are added for cross-reference and are not part of the code.

A.1 index.html

```
 1  <!doctype html>
 2  <html lang="en">
 3    <head>
 4      <meta charset="UTF-8" />
 5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 6      <title>Photo Editor</title>
 7      <link
 8        href="https://cdn.jsdelivr.net/npm/remixicon@4.9.0/fonts/remixicon.css"
 9        rel="stylesheet"
10      />
11      <link rel="stylesheet" href="theme.css" />
12      <link rel="stylesheet" href="style.css" />
13    </head>
14
15    <body>
16      <main>
17        <section>
18          <div class="left">
19            <div class="top">
20              <label for="image-input" class="btn"
21                >Choose Image
22                <input id="image-input" type="file" accept="image/*" hidden />
23              </label>
24              <button class="btn" id="reset-btn">Reset</button>
25              <button class="btn" id="download-btn">Download</button>
26            </div>
27            <div class="bottom">
28              <div class="placeholder">
29                <i class="ri-image-line"></i>
30                <p>No Image Chosen</p>
31              </div>
32              <canvas id="image-canvas"></canvas>
33            </div>
34          </div>
35          <div class="right">
36            <h2>Filters</h2>
37
38            <div class="filters">
39
40            </div>
41            <h2>Presets</h2>
42            <div class="presets"></div>
43
44          </div>
45        </section>
46      </main>
47      <script src="script.js"></script>
48    </body>
49  </html>
```

A.2 theme.css

```css
:root {
  /* bg colors */
  --bg-primary-color: rgb(26, 26, 26);
  --bg-secondary-color: rgb(103, 97, 97);
  --bg-active-color: rgb(240, 243, 247);
  --bg-danger-color: rgb(190, 30, 30);
  --bg-success-color: rgb(39, 172, 39);
  --preset-button-color: #343a40;

  /* text colors */
  --text-primary-color: rgb(255, 255, 255);
  --text-secondary-color: rgb(103, 97, 97);
  --text-tertiary-color: rgb(240, 243, 247);
  --text-active-color: rgb(0, 0, 0);

  /* padding */
  --padding-sm: 0.5rem;
  --padding-md: 1rem;
  --padding-lg: 1.5rem;
  --padding-xl: 2rem;
  --padding-2xl: 3rem;

  /* margin */
  --margin-sm: 0.5rem;
  --margin-md: 1rem;
  --margin-lg: 1.5rem;
  --margin-xl: 2rem;
  --margin-2xl: 3rem;

  /* gutter */
  --gutter-sm: 0.5rem;
  --gutter-md: 1rem;
  --gutter-lg: 1.5rem;
  --gutter-xl: 2rem;
  --gutter-2xl: 3rem;

  /* font sizes */
  --font-size-sm: 0.875rem;
  --font-size-md: 1rem;
  --font-size-lg: 1.25rem;
  --font-size-xl: 1.5rem;
  --font-size-2xl: 2rem;

  /* border radius */
  --border-radius-sm: 0.2rem;
  --border-radius-md: 0.25rem;
  --border-radius-lg: 0.3rem;
  --border-radius-xl: 0.4rem;
  --border-radius-2xl: 0.5rem;
}
```

A.3 style.css

```css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html,
body {
  width: 100%;
}

html {
  font-family:
    system-ui,
    -apple-system,
    BlinkMacSystemFont,
    "Segoe UI",
    Roboto,
    Oxygen,
    Ubuntu,
    Cantarell,
    "Open Sans",
    "Helvetica Neue",
    sans-serif;

  background-color: var(--bg-primary-color);
  color: var(--text-primary-color);
}

main {
  width: 100%;
}

main section {
  width: 100%;
  padding: var(--padding-2xl);
  display: flex;
  gap: var(--gutter-lg);
}

.btn {
  padding: var(--padding-md) var(--padding-lg);
  border: none;
  outline: none;
  border-radius: var(--border-radius-md);
  background-color: var(--bg-active-color);
  cursor: pointer;
  color: var(--text-active-color);
}

#reset-btn {
  background-color: var(--bg-danger-color);
  color: var(--text-primary-color);
}

#download-btn {
  background-color: var(--bg-success-color);
  color: var(--text-primary-color);
}

main section .left {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: var(--gutter-lg);
  width: 70%;
}

main section .right {
  width: 30%;
  background-color: var(--bg-secondary-color);
  padding: var(--padding-lg);
  border-radius: var(--border-radius-lg);
}

main section .top {
  display: flex;
```

```
  align-items: center;
  gap: var(--gutter-md);
}

main section .bottom {
  background-color: var(--bg-secondary-color);
  width: 100%;
  padding: var(--padding-2xl);
  aspect-ratio: 5/3;
  border-radius: var(--border-radius-xl);
  display: flex;
  justify-content: center;
  align-items: center;
}

main section .bottom .placeholder {
  display: flex;
  flex-direction: column;
  align-items: center;
  opacity: 0.6;
  gap: var(--gutter-md);
}

main section .bottom .placeholder i {
  font-size: 10rem;
}

main section .bottom .placeholder p {
  font-size: var(--font-size-lg);
}

main section .right {
  display: flex;
  flex-direction: column;
  gap: var(--gutter-md);
}

main section .right .filters {
  display: flex;
  flex-direction: column;
  gap: var(--gutter-md);
}

main section .right .filter {
  width: 100%;
}

main section .right .filter input {
  width: 100%;
  outline: none;
  margin-top: var(--margin-sm);
  appearance: none;
}

main section .right .filter input::-webkit-slider-runnable-track {
  background-color: var(--bg-active-color);
  height: 3px;
  border-radius: var(--border-radius-sm);
}

main section .right .filter input::-moz-range-track {
  background-color: var(--bg-active-color);
  height: 3px;
}

main section .right .filter input::-webkit-slider-thumb {
  cursor: pointer;
  appearance: none;
  width: 20px;
  height: 20px;
  border-radius: 50%;
  background-color: red;
  margin-top: -8.5px;
}

canvas {
  max-width: 90%;
  max-height: 80vh;
  display: none;
```

```
}

.presets {
  display: flex;
  flex-wrap: wrap;
  gap: var(--gutter-sm);
}

.preset-btn {
  padding: var(--padding-sm);
  border-radius: var(--border-radius-lg);
  background-color: var(--preset-button-color);
  color: var(--text-primary-color);
}
```

A.4 script.js

```
let filters = {
  brightness: {
    value: 100,
    min: 0,
    max: 200,
    unit: "%",
  },

  contrast: { value: 100, min: 0, max: 200, unit: "%" },

  saturation: {
    value: 100,
    min: 0,
    max: 200,
    unit: "%",
  },

  Huerotation: {
    value: 0,
    min: 0,
    max: 360,
    unit: "deg",
  },

  blur: {
    value: 0,
    min: 0,
    max: 20,
    unit: "px",
  },

  grayscale: {
    value: 0,
    min: 0,
    max: 100,
    unit: "%",
  },

  sepia: {
    value: 0,
    min: 0,
    max: 100,
    unit: "%",
  },

  opacity: {
    value: 100,
    min: 0,
    max: 100,
    unit: "%",
  },

  invert: {
    value: 0,
    min: 0,
    max: 100,
    unit: "%",
  },
};

const imageCanvas = document.querySelector("#image-canvas");
const imageInput = document.querySelector("#image-input");
const canvasCtx = imageCanvas.getContext("2d");
const resetBtn = document.querySelector("#reset-btn");
const downloadBtn = document.querySelector("#download-btn");
let file = null;
let image = null;

const filersContainer = document.querySelector(".filters");
const presetsContainer = document.querySelector(".presets");

function createFilterElement(name, unit = "%", value, min, max) {
  const div = document.createElement("div");
  div.classList.add("filter");

  const input = document.createElement("input");
  input.type = "range";
```

```
  input.min = min;
  input.max = max;
  input.value = value;
  input.id = name;

  const p = document.createElement("p");
  p.innerText = name;

  div.appendChild(p);
  div.appendChild(input);

  input.addEventListener("input", (event) => {
    filters[name].value = input.value;
    applyFilters();
  });
  return div;
}

function createPresetElement(name, preset) {}

function createFilters() {
  Object.keys(filters).forEach((key) => {
    const filterElement = createFilterElement(
      key,
      filters[key].unit,
      filters[key].value,
      filters[key].min,
      filters[key].max,
    );

    filersContainer.appendChild(filterElement);
  });
}

createFilters();

imageInput.addEventListener("change", (event) => {
  file = event.target.files[0];
  const imagePlaceholder = document.querySelector(".placeholder");
  imageCanvas.style.display = "block";
  imagePlaceholder.style.display = "none";

  const img = new Image();
  img.src = URL.createObjectURL(file);

  img.onload = () => {
    image = img;
    imageCanvas.width = img.width;
    imageCanvas.height = img.height;
    canvasCtx.drawImage(img, 0, 0);
  };
});

function applyFilters() {
  if (!image) return;

  canvasCtx.clearRect(0, 0, imageCanvas.width, imageCanvas.height);

  canvasCtx.filter = `
    brightness(${filters.brightness.value}${filters.brightness.unit})
    contrast(${filters.contrast.value}${filters.contrast.unit})
    saturate(${filters.saturation.value}${filters.saturation.unit})
    hue-rotate(${filters.Huerotation.value}${filters.Huerotation.unit})
    blur(${filters.blur.value}${filters.blur.unit})
    grayscale(${filters.grayscale.value}${filters.grayscale.unit})
    sepia(${filters.sepia.value}${filters.sepia.unit})
    opacity(${filters.opacity.value}${filters.opacity.unit})
    invert(${filters.invert.value}${filters.invert.unit})
  `;

  canvasCtx.drawImage(image, 0, 0);
}

resetBtn.addEventListener("click", () => {
  filters = {
    brightness: {
      value: 100,
      min: 0,
      max: 200,
```

```
      unit: "%",
    },

    contrast: { value: 100, min: 0, max: 200, unit: "%" },

    saturation: {
      value: 100,
      min: 0,
      max: 200,
      unit: "%",
    },

    Huerotation: {
      value: 0,
      min: 0,
      max: 360,
      unit: "deg",
    },

    blur: {
      value: 0,
      min: 0,
      max: 20,
      unit: "px",
    },

    grayscale: {
      value: 0,
      min: 0,
      max: 100,
      unit: "%",
    },

    sepia: {
      value: 0,
      min: 0,
      max: 100,
      unit: "%",
    },

    opacity: {
      value: 100,
      min: 0,
      max: 100,
      unit: "%",
    },

    invert: {
      value: 0,
      min: 0,
      max: 100,
      unit: "%",
    },
  };

  applyFilters();

  filersContainer.innerHTML = "";
  createFilters();
});

downloadBtn.addEventListener("click", () => {
  const link = document.createElement("a");
  link.download = "edited-image.png";
  link.href = imageCanvas.toDataURL();
  link.click();
});

const presets = {
  normal: {
    brightness: 100,
    contrast: 100,
    saturation: 100,
    Huerotation: 0,
    blur: 0,
    grayscale: 0,
    sepia: 0,
    opacity: 100,
    invert: 0,
```

```
  },

  drama: {
    brightness: 90,
    contrast: 145,
    saturation: 125,
    Huerotation: 0,
    blur: 0,
    grayscale: 10,
    sepia: 5,
    opacity: 100,
    invert: 0,
  },

  vintage: {
    brightness: 105,
    contrast: 90,
    saturation: 75,
    Huerotation: 350,
    blur: 0.3,
    grayscale: 10,
    sepia: 35,
    opacity: 100,
    invert: 0,
  },

  oldSchool: {
    brightness: 95,
    contrast: 110,
    saturation: 65,
    Huerotation: 0,
    blur: 0,
    grayscale: 25,
    sepia: 45,
    opacity: 100,
    invert: 0,
  },

  cinematic: {
    brightness: 85,
    contrast: 130,
    saturation: 85,
    Huerotation: 345,
    blur: 0,
    grayscale: 15,
    sepia: 10,
    opacity: 100,
    invert: 0,
  },

  blackAndWhite: {
    brightness: 105,
    contrast: 130,
    saturation: 0,
    Huerotation: 0,
    blur: 0,
    grayscale: 100,
    sepia: 0,
    opacity: 100,
    invert: 0,
  },

  faded: {
    brightness: 115,
    contrast: 80,
    saturation: 70,
    Huerotation: 0,
    blur: 0,
    grayscale: 5,
    sepia: 15,
    opacity: 100,
    invert: 0,
  },

  cool: {
    brightness: 100,
    contrast: 110,
    saturation: 110,
    Huerotation: 190,
```

```
315       blur: 0,
316       grayscale: 0,
317       sepia: 0,
318       opacity: 100,
319       invert: 0,
320     },
321
322     warm: {
323       brightness: 105,
324       contrast: 105,
325       saturation: 115,
326       Huerotation: 350,
327       blur: 0,
328       grayscale: 0,
329       sepia: 20,
330       opacity: 100,
331       invert: 0,
332     },
333
334     moody: {
335       brightness: 75,
336       contrast: 135,
337       saturation: 70,
338       Huerotation: 210,
339       blur: 0,
340       grayscale: 15,
341       sepia: 5,
342       opacity: 100,
343       invert: 0,
344     },
345
346     fadedFilm: {
347       brightness: 110,
348       contrast: 85,
349       saturation: 80,
350       Huerotation: 5,
351       blur: 0.2,
352       grayscale: 10,
353       sepia: 25,
354       opacity: 100,
355       invert: 0,
356     },
357
358     dreamy: {
359       brightness: 115,
360       contrast: 90,
361       saturation: 110,
362       Huerotation: 340,
363       blur: 1,
364       grayscale: 0,
365       sepia: 5,
366       opacity: 100,
367       invert: 0,
368     },
369
370     noir: {
371       brightness: 80,
372       contrast: 160,
373       saturation: 0,
374       Huerotation: 0,
375       blur: 0,
376       grayscale: 100,
377       sepia: 0,
378       opacity: 100,
379       invert: 0,
380     },
381
382     retro: {
383       brightness: 105,
384       contrast: 95,
385       saturation: 85,
386       Huerotation: 10,
387       blur: 0,
388       grayscale: 15,
389       sepia: 30,
390       opacity: 100,
391       invert: 0,
392     },
393
```

```
  cyberpunk: {
    brightness: 95,
    contrast: 140,
    saturation: 160,
    Huerotation: 280,
    blur: 0,
    grayscale: 0,
    sepia: 0,
    opacity: 100,
    invert: 0,
  },

  soft: {
    brightness: 110,
    contrast: 90,
    saturation: 95,
    Huerotation: 0,
    blur: 0.5,
    grayscale: 0,
    sepia: 0,
    opacity: 100,
    invert: 0,
  },
};

Object.keys(presets).forEach((presetName) => {
  const presetbtn = document.createElement("button");
  presetbtn.classList.add("preset-btn");
  presetbtn.innerText = presetName;
  presetsContainer.appendChild(presetbtn);

  presetbtn.addEventListener("click", () => {
    const preset = presets[presetName];
    Object.keys(preset).forEach((filterName) => {
      filters[filterName].value = preset[filterName];
    });

    applyFilters();
  });
});
```